Lab: Creating Architecture Design Document

Estimated time: **30 minutes**

Overview

In this lab, you critically analyze the architectural structure of the Smart Clinic Management System. This system uses both **Spring MVC with Thymeleaf** for admin/doctor dashboards and **REST APIs** for other modules. It also interacts with **two databases**—MySQL and MongoDB—through service and repository layers.

Key tasks

As part of this lab, you perform the following tasks:

- Study the reference diagram provided in this lab and understand the flow of data and control
- Based on your understanding, you will create an architecture description of the Smart Clinic Management System using numbered steps and labeled components

This lab will help you understand how frontend components connect with backend logic, how services and repositories interact, and how data flows through the full stack.

Learning objectives

By the end of this lab, you will be able to:

- Identify the core components of a Spring Boot application architecture
- Describe how MVC and REST modules coexist
- Trace the flow of data and control through controllers, services, and repositories
- Explain how dual databases are integrated into a single service layer
- Document and communicate architectural decisions clearly

Architecture design overview

Three-tier web application design

The Smart Clinic Management System follows a **three-tier web architecture**, which separates the system into three distinct layers:

1. **Presentation Tier** – The user interface, consisting of Thymeleaf templates and REST API consumers
2. **Application Tier** – The Spring Boot backend that contains the controllers, services, and business logic
3. **Data Tier** – The databases: MySQL for structured data and MongoDB for flexible, document-based data

This structure improves scalability, maintainability, and deployment flexibility. Each tier can be independently developed, tested, or scaled without impacting the others directly.

Why Spring Boot and this tech stack

We chose **Spring Boot** as the foundation for this application because it simplifies backend development while enforcing best practices. It supports:

- **Spring MVC** for server-rendered admin and doctor dashboards
- **REST APIs** for modular and scalable client-server communication
- **Spring Data JPA** for interacting with a MySQL database
- **Spring Data MongoDB** for storing flexible prescription records

Spring Boot is developer-friendly, production-ready, and integrates easily with testing, validation, and containerization tools.

REST APIs for scalable integration

For modules like appointments, patient dashboards, and patient records, we expose **RESTful APIs** instead of using server-side views. REST allows external clients—such as mobile apps or future web apps—to communicate with the backend via lightweight HTTP and JSON.

This makes the system more extensible and interoperable, supporting real-time client applications, third-party integrations, and cross-platform access.

Deployability and CI/CD compatibility

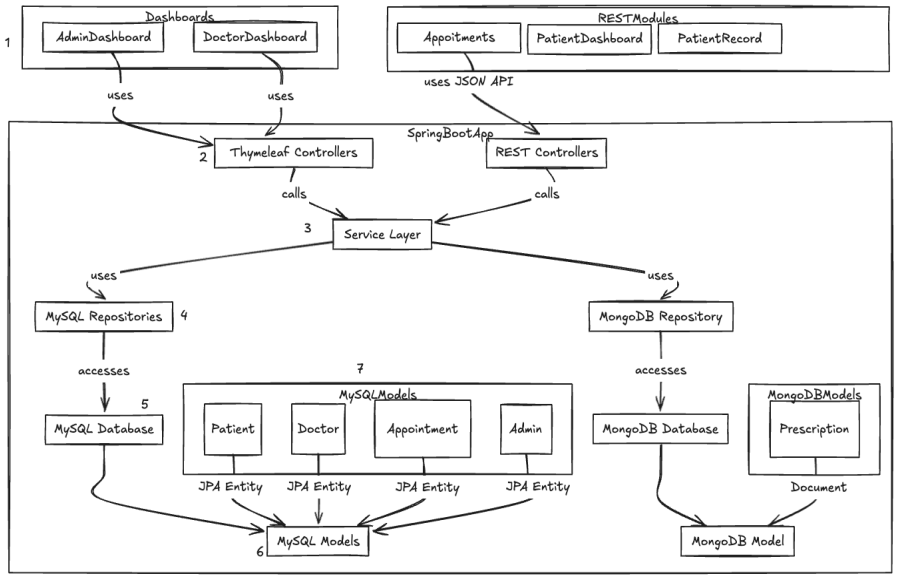
Spring Boot applications are easily containerized using Docker, making them ideal for deployment in cloud-native environments like Kubernetes or virtual machines. They start quickly, run reliably across systems, and scale horizontally as needed.

The system also integrates well with modern **CI/CD pipelines** (such as GitHub Actions, Jenkins, or GitLab CI), enabling automated builds, tests, and deployments. This improves release speed, reduces manual errors, and supports continuous delivery of new features and fixes in production.

Reference diagram

Below is a simplified architecture diagram of the Smart Clinic Management System. This system:

- Renders HTML via Thymeleaf for the admin and doctor dashboards
- Exposes JSON-based REST APIs for appointment and patient modules
- Connects to **MySQL** for structured data (Patient, Doctor, Appointment, Admin)
- Connects to **MongoDB** for unstructured document-based data (Prescription)



Each numbered arrow represents a key step in the flow of control or data.

Step-by-step architecture walkthrough

This section provides a detailed walkthrough of how the Smart Clinic application is structured, how requests flow through it, and how different technologies interact across the layers. The architecture follows a clean separation of concerns and adheres to Spring Boot best practices.

Each step in the diagram represents a specific tier or logical component in the application and plays a key role in processing requests, applying business logic, and persisting or retrieving data.

1. User Interface Layer

The system supports multiple user types and interaction patterns. Users can access the application through:

- **Thymeleaf-based web dashboards** such as `AdminDashboard` and `DoctorDashboard`. These are traditional HTML pages rendered on the server and delivered to the browser.
- **REST API clients** like mobile apps or frontend modules (e.g., `Appointments`, `PatientDashboard`, and `PatientRecord`) that interact with the backend via HTTP and receive JSON responses.

This separation allows the system to support both interactive browser views and scalable API-based integrations.

2. Controller Layer

When a user interacts with the application (e.g., clicking a button or submitting a form), the request is routed to a backend **controller** based on the URL path and the HTTP method.

- Requests for server-rendered views are handled by **Thymeleaf Controllers**, which return `.html` templates that will be filled with dynamic data and rendered in the browser.
- Requests from API consumers are handled by **REST Controllers**, which process the input, call backend logic, and return responses in JSON format.

These controllers serve as the entry points into the backend application logic, enforcing request validation and coordinating the request/response flow.

3. Service Layer

All controllers delegate logic to the **Service Layer**, which acts as the heart of the backend system. This layer:

- Applies business rules and validations
- Coordinates workflows across multiple entities (e.g., checking doctor availability before scheduling an appointment)
- Ensures a clean separation between controller logic and data access

By isolating business logic here, the application becomes more maintainable, testable, and easier to scale.

4. Repository Layer

The service layer communicates with the **Repository Layer** to perform data access operations. This layer includes two types of repositories:

- **MySQL Repositories**, which use Spring Data JPA to manage structured relational data like patients, doctors, appointments, and admin records.
- **MongoDB Repository**, which uses Spring Data MongoDB to manage document-based records like prescriptions.

Repositories abstract the database access logic and expose a simple, declarative interface for fetching and persisting data.

5. Database access

Each repository interfaces directly with the underlying database engine:

- **MySQL** stores all core entities that benefit from a normalized relational schema and constraints—such as users, roles, and appointments.
- **MongoDB** stores flexible and nested data structures, such as prescriptions, which may vary in format and allow for rapid schema evolution.

This dual-database setup leverages the strengths of both structured and unstructured data storage approaches.

6. Model binding

Once data is retrieved from the database, it is mapped into Java model classes that the application can work with. This process is known as **model binding**.

- In the case of MySQL, data is converted into **JPA entities**, which represent rows in relational tables and are annotated with `@Entity`.
- For MongoDB, data is loaded into **document objects**, typically annotated with `@Document`, which map to BSON/JSON structures in collections.

These model classes provide a consistent, object-oriented representation of the data across the application layers.

7. Application models in use

Finally, the bound models are used in the response layer:

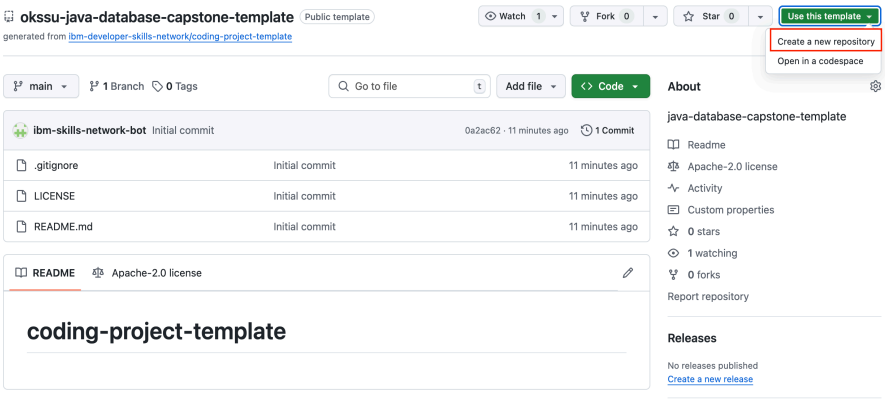
- In **MVC flows**, models are passed from the controller to Thymeleaf templates, where they are rendered as dynamic HTML for the browser.
- In **REST flows**, the same models (or transformed DTOs) are serialized into **JSON** and sent back to the client as part of an HTTP response.

This marks the end of the request-response cycle, delivering either a full web page or structured API data, depending on the consumer.

Your task: Create repository from template

1. Open a browser and visit the following GitHub template repository:
   https://github.com/ibm-developer-skills-network/okssu-java-database-capstone-template

2. Click the **Use this template** drop-down menu and click **Create a new repository.**



3. On the resulting screen:

   - Select your GitHub account from the dropdown.
   - **Name the new repository**: `java-database-capstone`
     (This is important so that automated scripts in future labs work correctly.)
   - Optionally add a description like "Capstone project for Java development with databases."
   - Make sure the repo is set to **Public** to make your repository visible to anyone on the internet.
   - Click **Create repository.**

Create a new reposi

A repository contains all project fi
elsewhere? Import a repository.

Required fields are marked with a

Repository template

ibm-developer-skills-netw

Start your repository with a template r

☐ Include all branches
Copy all branches from ibm-devel
branch.

Owner *

[redacted]

Repo

/ java

java

Great repository names are short

Description (optional)

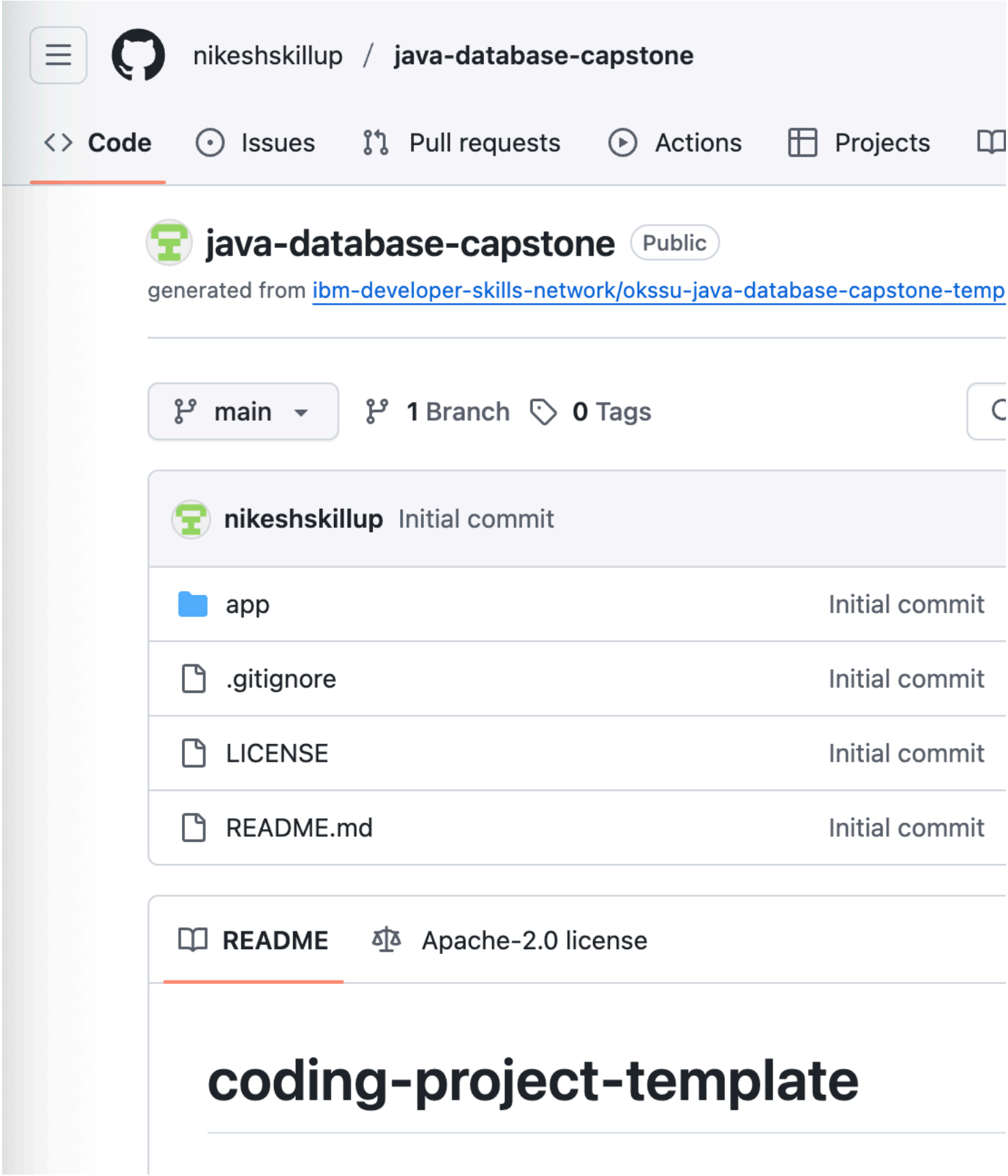
Public
Anyone on the internet can

Private
You choose who can see ar

You are creating a public repo

Task 2: Clone the repository

1. Click the **Code** button and copy the `https` URL.



2. Switch to your **Skills Network Labs** Cloud IDE tab.
3. Open a new terminal by selecting `Terminal > New Terminal`.
4. Clone the repository using:

```
git clone <url-you-copied>
```

Your task: Document your architecture

In your GitHub repo (`java-database-capstone`), create a new file in the base directory:

```
schema-architecture.md
```

This file will contain two sections:

Section 1: Architecture summary

Write a paragraph or two describing your application's architecture in your own words. You may use this template and modify it:

> This Spring Boot application uses both MVC and REST controllers. Thymeleaf templates are used for the Admin and Doctor dashboards, while REST APIs serve all other modules. The application interacts with two databases—MySQL (for patient, doctor, appointment, and admin data) and MongoDB (for prescriptions). All controllers route requests through a common service layer, which in turn delegates to the appropriate repositories. MySQL uses JPA entities while MongoDB uses document models.

Section 2: Numbered flow of data and control

Create a numbered list (1 to 7) describing each step of the data flow based on the architecture diagram. Use your own words or base your summary on the explanation above. You do not have to include the architectural diagram itself.

Example format:

```
1. User accesses AdminDashboard or Appointment pages.
2. The action is routed to the appropriate Thymeleaf or REST controller.
3. The controller calls the service layer...
...
```

Save and push your work

After completing your Markdown file, make sure you commit it back to your GitHub repository.

If you're using GitHub in the browser:

1. Open your GitHub repository: `java-database-capstone`
2. In the base directory, click **Add file → Create new file**
3. Name the file: `schema-architecture.md`
4. Paste your content and scroll down
5. Add a commit message like:

```
Added architecture design for Smart Clinic app
```

6. Commit the file directly to the main branch

If you are not using the browser and using the lab environment for the git operations:

1. Save your `schema-architecture.md` file
2. Stage and commit your changes:

```
git add schema-architecture.md
git commit -m "Added schema architecture for Smart Clinic"
```

3. Push to your GitHub repo:

```
git push https://<your_username>:<your_token>@github.com/<your_username>/java-database-capstone.git
```

Conclusion

Congratulations! You've articulated and documented the architecture of the Spring Boot application. The learning from this lab will help you build and debug the system, and it sets a strong foundation for collaborating with others.

In subsequent labs, you will implement this architecture using models, controllers, and repositories, exactly as described in this design.

Author(s)

IBM Skills Network Team
Upkar Lidder